

Devolución TP2

Grupo 1

Demo

- Volver a eventos no vuelve realmente a eventos, sino que hace un back (lo que podría dejarte en la misma pantalla por ejemplo si te inscribiste)
- Las etiquetas no son clickeables para ir a buscar otros eventos
- El search por tag no tiene autocomplete, pese a existir para la carga de eventos
- Los eventos se muestran en rows de 3... pero se muestran 8 por página?
- El popup "glass" con texto verde es muy difícil de leer por falta de contraste
- Me inscribí al evento `<script>alert(22)</script>` y me dice "pendiente". No lo veo en mi perfil en ningún lado.
- La sección de "Para ti" no considera si ya estás o no inscrito al evento.

Código

- Omiten modificadores de acceso y no parece ser por diseño.
- Hay clases con muchísima lógica (600+ LoCs) que podría ser simplificada o refactorizada.
- Corrigen los errores de la primer entrega.
- Implementan correctamente paginación.

Seguimiento

- No resta puntos por seguimiento

Nota

8; promedio 7

Grupo 2

Demo

- No puedo usar un room si el room tiene una estadía que no fue marcada como completa, sin embargo no me dice que tengo marcarla como completa para liberar el room. El flujo es complicado de seguir sin haberlo usado previamente.
- Hay datos inconsistentes de reservas previas.
- Hay que aclarar los años en las fechas de los bookings.
- El botón de back después de hacer un booking quiere reenviar la info del booking otra vez.
- El go back to the home page hace un back en stack en vez de dirigir a la home.
- Los comentarios no están paginados.

Código

- Corrigen los problemas detectados en la primer entrega en forma satisfactoria.
- Loop Infinito en `remindUpcomingBookingsCronJob` que causa el envío de infinitos mails a un mismo usuario. **Esto es un error grave.**
- En el método `getUserBookings`, hay una condición incorrecta en la subquery que causa que no se respete el límite de bookings indicado (`WHERE pinner.id < pinner.id`). **Esto es un error grave que denota falta de cobertura en tests unitarios.**
- Métodos (como `sendBookingCancellationMessages`) marcados como `@Async` y `@Transactional` acceden a relaciones lazy sin materialización explícita. Deberían materializar explícitamente las relaciones lazy antes de usarlas en el contexto async, o usar fetch joins en las queries del DAO. **Esto es un error conceptual.**

```
bookings.forEach(booking → {  
    booking.getUser().getId(); // Forzar inicialización  
    messageSender.sendBookingCancellationMessage(booking.getUser  
(), booking);  
});
```

Seguimiento

- No resta puntos por seguimiento.

Nota

7; promedio 7

Grupo 3

Demo

- Me deja registrarme con email vacío.
- Dejan subir fotos como recursos y luego se no se pueden visualizar.
- Caracteres especiales no están bien manejados: "ÃÂÃÂÃÂÃÂÃÂ" al editar una clase.
- Al generar un error los tags se pierden
- El explore está incompleto para un producto cuyo objetivo tiene centralizar y hacer más simple la obtención de contenido educativo.
 - El search borra los filtros
 - Al sistema de tags le falta un poquito, no se puede explorar y no están ordenados por ningún parámetro en particular. Como está ahora presenta poca utilidad.
 - No hay ordenamientos interesantes por sobre rating
- No puedo elegir lessons porque el selector no está paginado por ende quedo afuera de esa funcionalidad de la página si las lessons que quiero elegir no entraron en la página 1.
- Faltó el C:OUT en el selector de lesson
- Falta paginación de los hijos de los comentarios
- Problemas de UX
 - La paginación no es consistente
 - Son susceptibles a doble click, prácticamente en toda la página.
 - Elementos que aparentan ser clickeables no lo son

Código

- Rutas de suscripción / de-suscripción sin protección explícita. La verificación `if (loggedUser != null)` es manual. Debería estar protegida por Spring Security. **Esto es una vulnerabilidad grave que ya había sido marcada en la primer entrega.**
- Se accede a objetos Optional con `.get()` sin previa verificación de ausencia. **Esto ya había sido marcado en la primer entrega.**
- No se utiliza estrategia 1+1 para la paginación. **Esto es un error conceptual grave.**
- Hay colecciones que pueden crecer desmedidamente (ej: `comments`) mapeadas via `toMany` que provocan queries no optimizadas que traen a memoria cantidades posiblemente grandes de recursos sin paginar.
- Muy buena cobertura de tests
- Hay un `HomeController` y un `home.jsp` viejos que no se utilizan.
- Dependencia hsqldb sin scope test en módulo persistence. **Esto es un error conceptual.**
- Usan `.clear()` que materializa toda la lista de elementos para hacer una simple modificación (ej: en tags).
- Métodos marcados como `@Async` y `@Transactional` acceden a relaciones lazy sin materialización explícita. Deberían materializar explícitamente las relaciones lazy antes de usarlas en el contexto async, o usar fetch joins en las queries del DAO

Seguimiento

- No resta puntos por seguimiento.

Nota

4; promedio 5

Grupo 4

Demo

- Si cambio el idioma del browser se queda en ingles
- Esta raro el hover de trips que muestra el dropdown cuando no debería
- En la sección de perfil los request/process/completed podrían ser clickeables
- 400 Bad request custom bien
- Esta bueno el dashboard del perfil del fletero
- Me deja cargarle imágenes/ editar a los camiones de otros fleteros
- Puedo asignar camiones de otras empresas a mis drivers
- No hay filtro de UTF-8

Código

- Incluyen la dependencia sobre `hsqldb` en `compile` en el pom de webapp, aunque el pom padre y el de persistence lo declaran en scope test. **Esto es un error conceptual.**
- La moneda no esta internacionalizada en los JSP.
- En lugar de utilizar el `SessionLocaleResolver`, podrían utilizar `AcceptHeaderLocaleResolver` ya que la implementación actual impide que se cambie el idioma.
- `getCurrentUserId` en `ControllerUtils` debería ser un `@ModelAttribute`
- Los try catch de las exceptions en los controllers podrían ser manejadas por el `ExceptionHandlerController`
- La utilización de `session.getAttribute("emailForRegistration")` es un poco confusa ya que implica que en un paso previo debió ser seteado. Lo mejor seria agregar este dato a los Forms como un input hidden.
- La lógica de validación de input en `uploadReceipt` podría ser simplificada en un custom validator
- Las clases utilitarias deberían ser final y no instanciables. `ControllerUtils` no es final y solo tiene un constructor package-private
- No aplican el modelo de consulta 1+1. **Esto es un error conceptual grave.**
- Hacen uso de fetch type EAGER en MUCHOS modelos, esto no es recomendado ya que al cargar 20 ofertas en una lista paginada, se pueden

ejecutar ~121 queries en lugar de 1. Lo mejor seria tener fetch type lazy y crear algún método con JOIN FETCH (teniendo en cuenta los recaudos necesario) para los métodos que requieran los datos. Además no siempre hacen uso de todas las entidades. **Esto es un error conceptual grave**

- La entidad image trae siempre los bytes de la imagen
- Hacen mal uso de `JOIN FETCH c.transportes` esto genera que si un flete tiene 20 conductores y cada uno tiene 5 transportes, se cargan 100 transportes en una sola query. **Esto es un error conceptual grave**
- Usan `.add()` y `.remove()` en las Locations, esto materializa toda la lista de locations para hacer una simple modificación. Lo mismo sucede con las imágenes (que es pero porque trae todos los bytes).
- Hay tests que no son unitarios (utilizan mas de un método de la class under test). **Esto es un error conceptual grave.**
- Utilizan `mockito.verify` , esto valida la implementación y no el comportamiento. **Esto es un error conceptual grave.**

Seguimiento

- No resta puntos por seguimiento.

Nota

4; promedio 4.5

Grupo 5

Demo

- Bien por poder volver desde el login a la home
- 400 y 500 custom bien
- el modal de confirmar compra tiene 2 steps, es un poco friccionante tener que hacer 3 click para comprar algo, seria mejor que el modal te muestre toda la info de una y que luego de 1 click dentro del modal ya proceda a comprar
- Validation para que no te deje enviar un comprobante vacío

- Si subo una imagen como comprobante y toco ver imagen me abre una vista con un PDF preview que se rompe porque no es un PDF
- Estaría bueno que haya un modal para subir el comprobante, resulta raro que haya un input con un botón de submit
- El campo country debería ser un autocomplete y no un input libre
- Si voy a la url de editar un vinilo que no es mio no hay nada que me frene y puedo editarlo

Código

- Tiene algunas definiciones de versiones de dependencias en los poms hijos, repitiendo o cambiando la del pom padre. **Esto ya había sido marcado en la primer entrega.**
- Setean java 11 y 15 en el plugin de compile de services y persistence en lugar de usar el que se hereda del padre.
- En lugar de hacer `extractVinylIdFromUri(uri);` podrían sumar el vinyl ID directamente en la exception para evitar hacer un parseo de la URI en el `GlobalExceptionHandler` **Esto ya había sido marcado en la primer entrega**
- Hay versiones en el pom padre que no establecen la versión como una variable, si bien no es obligatorio es una buena practica realizarlo.
- No tienen en el repositorio el archivo logback-test sino que únicamente el de producción.
- `UserController` tiene algo de lógica de negocio. **Esto es un error conceptual.**
 - `becomeSeller` actualiza la info del user pero el rol se agrega en en controller, debería ser parte de `userService.updateUserInfo`
 - Además `if(roleService.userHasRole(currentUser.getId(), "SELLER"))` debería ser un chequeo de security y no lógica del controller
 - `/resetPassword` hace uso de `emailService.sendPasswordResetEmail` atando el hecho de que se envía un mail para resetear la contraseña, si en un futuro se añadiesen otros métodos deberían ser agregados en el controller con esta lógica, lo cual es incorrecto. Lo correcto es llamar a un service que internamente llame al email service.
- Métodos marcados como `@Async` y `@Transactional` acceden a relaciones lazy sin materialización explícita. Deberían materializar explícitamente las

relaciones lazy antes de usarlas en el contexto async, o usar fetch joins en las queries del DAO

- La query de `similarVinyls` tiene un error en la precedencia

`WHERE v.publicationStatus = 'ACTIVE' AND v.stock > 0 AND v.genre = :g OR v.sellerId = :sellerId` se evalúa como:

`WHERE (v.publicationStatus = 'ACTIVE' AND v.stock > 0 AND v.genre = :g) OR v.sellerId = :sellerId` Es decir trae cualquier vinilo del seller sin importar que este active o stock > 0 (inclusive el mismo vinilo podría venir). **Esto es un error grave.**

- Hay algunos tests que no hacen `em.flush` y por ende la validación del estado de la db no es correcta.

Seguimiento

- No resta puntos por seguimiento.

Nota

6; promedio 6.5

Grupo 6

Demo

- Validar tu cuenta dice que vayas a loguearte, pero el usuario ya está logueado
- La reserva muestra su UUID... para el usuario esto es usable?
- por qué el dropdown de fechas muestra aquellas que no tienen disponibilidad?
- El mail de reserva confirmada en español dicen "Hola {0}"
 - Este mail te indica a "mostrar tu código"... qué código?
 - El mail dice que mi reserva está confirmada, aunque yo todavía no pagué... el mail no menciona que falte un pago.

Código

- El módulo interfaces declarar depender sobre `spring-web` ... Las capas de servicio y persistencia son agnósticas del contexto web, no debieran depender de esto para nada. **Esto es un error conceptual grave que ya había sido marcado en la entrega anterior.**
- El repositorio incluye un directorio `out/artifacts` con artefactos autogenerados que no debieran ser parte del repositorio.
- El código hace referencia a una propiedad `security.rememberme.key` que no existe en ningún archivo de configuración.
- Hay bastante lógica de negocio en los controllers, ej: el edit club obtiene el club primero, evalúa que dato cambiar uno a uno, lo cambia y luego vuelve a llamar al servicio para editarlo. **Esto es un error conceptual grave que ya había sido marcado en la entrega anterior.**
- Son muy pocas las relaciones que están mapeadas, en la mayoría de los casos simplemente se mapean columnas con `UUID` s opacos. Esto imposibilita hacer cosas como queries en función del modelo de dominio y obliga a hablar en términos de la base de datos. **Esto es un error conceptual.**

Seguimiento

- Suma 0.5 por seguimiento de TP1

Nota

6.5; promedio 6

Grupo 7

Demo

- Revisar los botones de sysadmin, ojo con el encoding "â Volver al dashboard"
- Al tener un error en el formulario de creación de organización, deja de mostrar el botón de perfil y muestra el botón de login
- Los tags de las entidades no están internacionalizados

- Si se crean de forma dinámica, que se carguen a la base de datos con su traducción
- Falta una verificación de email y cambiar contraseña
- Me gustaría editar la organización como administrador
- El error de orden no válido al crear un puesto no tiene mensaje asignado
- En algunas páginas no hay favicon (<http://pawserver.it.itba.edu.ar/paw-2025b-07/org/2/open-position/12/manage?>)
- Los mails llegan en español siempre (no están internacionalizados).

Código

- Definen la versión de `commons-fileupload` en el pom de webapp.
- Mejoró la cobertura de tests pero faltan para servicios con lógica compleja como eventos y organigramas. Tampoco es correcto el uso de `Mockito.verify`, esto valida la implementación y no el comportamiento. **Esto es un error conceptual grave.**
- En la primer entrega se había marcado la ausencia de i18n. En esta entrega, aunque se han hecho grandes avances, **se ven todavía graves problemas:**
 - Tienen `lang="es"` hardcodeado en varios jsps.
 - Tienen mensajes en el `GlobalExceptionHandler` que no están internacionalizados.
 - Los valores de validaciones están hardcodeados en muchos mensajes de error localizados.
- Hay oportunidades perdidas de usar enums como request params, o en ciertos casos como commentPage ints, y así no tener que parsear ustedes los strings.
- Podrían usar la taglib de spring security para distinguir por los roles en los jsp.
- En `OrgController` está hardcodeado que siempre se ve la página 0.
- Tienen lógica de negocio en los controllers. Por ejemplo en `OrgCreateController`, el controller crea la organización y asigna el manager. En `OpenPositionManagementController` es el controller quien rechaza o reabre las aplicaciones ante un cambio. **Esto es un error conceptual grave.**

- Tienen funciones que devuelven colecciones de cardinalidad variable (e.g. enrollments, likes de un usuario, suscriptores) sin paginar. **Esto es un error grave.**
- Sus tests de persistencia suelen no ser unitarios, pues usan otros métodos del mismo u otros daos para el setup o check. **Esto es un error conceptual grave.**
- Tienen lógica de los servicios en los daos. Por ejemplo, que asignar el rol default USER implica borrar los roles o quien puede acceder a una imagen.
- Al manejar cantidades de likes o de comentarios como un campo calculado, terminan modificando la misma tabla desde varios daos. Podrían utilizar `@Formula`.
- Hacen uso extenso de join fetch. Positivamente, parece ser que siempre o terminan usando las entidades relacionadas o las aplican en un mapeo `@ManyToOne`.
- Podrían usar `@ManyToMany` y `@JoinTable` para mapear relaciones sin atributos.
- No siguen la convención de nombres de Java.

Seguimiento

- No resta puntos por seguimiento.

Nota

4; promedio 3

Grupo 8

Demo

- En la sección de reseñas, dice 1 reseñas
- En el mail se ve la información de cuándo se reservó pero no los detalles de la reserva, para eso hace falta ingresar a la página
- No queda evidente que tengo que cargar el comprobante

- Es válida la elección de volver al detalle del boliche para que sea más fácil seguir reservando, pero mostrar un mensaje
- Estaría bueno dejar al usuario con la sesión iniciada después del cambio de contraseña
- No es evidente que tengo que agregar mesas o eventos para que mi boliche sea descubrible, agregaría un aviso o mensaje

Código

- Corrigen los errores de la primera entrega.
- Podrían usar Request Params que no sean strings (e.g. para número de página, enums para sus criterios de orden), de esta forma delegando el parsing a spring y no realizarlo a mano.
- Si bien tienen validaciones previas, sería ideal que `saveReceipt` mantenga el estilo de los otros servicios y arroje excepción ante argumentos inválidos.
- Tienen algunas validaciones a nivel servicio (e.g. `createTable`) que estaría bueno tener como validaciones en los forms.
- En algunas queries paginadas tienen sentencias orderBy extras siendo que ordenan el resultado de la query.
- Estaría bueno que saquen los getters de listas de cardinalidad grande en las entidades(e.g. en Club), si bien no los usan.
- En sus TypedQueries, no usan distinct al obtener los ids para las queries 1+1.

Seguimiento

- No resta puntos por seguimiento.

Nota

8; promedio 8

Grupo 9

Demo

- En la pagina de login faltaría un header para saber qué pagina es
- En algunos mails les falta un header con información de la web. Los CTA están bien
- Al recuperar la contraseña estaría bueno que el usuario quede logueado
- Bien por recomendar duración de turnos
- El calendario esta bien hecho, lo único que le faltaría es poder borrar intervalos mas cortos y no el intervalo completo
- Tienen campos susceptibles a XSS (Coverage y plan). **Esto es un error grave.**
- No se puede poder ni precio de la consulta u obras sociales con las que se trabaja
- Si aprieto dos veces el pedir estudio se piden dos veces
- La aplicación es bastante completa y permite un buen flujo medico paciente desde el pedido del turno hasta la receta de prescripción

Código

- Para validar el estado de la base de datos utilizan `queryForMap` y `countRowsInTableWhere` . Sería deseable que utilicen uno solo de manera consistente.
- La cobertura de tests es muy completa.
- Corrigen satisfactoriamente las correcciones de la primera entrega.
- Tienen implementaciones de `findAll` sin una paginación adecuada. **Esto es un error grave.**

Seguimiento

- "Correcciones demo entrega" es una story muy general. Debiera haber sido descompuesta en distintas tareas.
- No resta puntos por seguimiento.

Nota

7; promedio 7

Grupo 10

Demo

- Mucha inconsistencia al manejo de errores de los forms
 - Llenar el form de swap con error quita lo que había puesto de fechas.
Esto es un error.
- Buena cantidad de Features
 - Buena UX
 - La feature de Trip y planear trip quedó bien
- Problemas pasados arreglados
- El explore quedó muy simple
 - Falta ordenamiento
 - Las cards no muestran reviews ni nro de comentarios, haciendo estas features poco útiles para la búsqueda.
 - Faltan amenities en el search

Código

- Corregido parcialmente: "Faltan **muchos** `@Transactional` [...]"
 - Queda: `RoomAvailabilityService` - método `addAvailability` hace insert sin `@Transactional`
 - Queda: `ImageServiceImpl` - métodos `upload` y `deleteImage` sin `@Transactional`
- La cobertura de tests es pobre
 - Hay DAOs grandes que no tienen tests
 - No cubren casos borde
 - No cubren casos no felices
- Hay mensajes hardcodeados sin i18n en el `GlobalExceptionHandler`
- No hacen uso del modelo 1+1. **Esto es un error conceptual grave.**
- Hay clases Utils no `final` o con constructor público

- Hay un mapeo de `Image` que es incorrecto y trae los bytes de la imagen cuando solo se necesita el id ya que el acceso se hace vía un endpoint separado

Seguimiento

- No resta puntos por seguimiento.

Nota

6; promedio 5.5

Grupo 11

Demo

- Se podría recomendar default utilizando reseñas en vez de hardcodeandolas
- Bien que se puede cancelar un trabajo porque dejan cambiar el precio
- No se muestra el slot elegido de una application
- El back de un Job no funciona
- Se muestran las reviews del usuario en el perfil del Job

Código

- Sigue habiendo lógica de negocio en algunos controllers, como por ejemplo en `JobApplicationController`, los métodos `canCancelApplication()`, `canCancelApplicationByOfferOwner()`, `canAcceptApplication()` están en el service pero deberían ser parte de la construcción de un DTO en el service, no llamados individualmente desde el controller. **Esto ya había sido marcado en la primer entrega.**
 - Otro ejemplo de esta corrección es en `declineApp`, no deberían primero buscar el jobApplication y luego pasarle el ID al mismo servicio, sino llamar al servicio una sola vez y que este realice las validaciones correspondientes.
 - Validación de email usando `userService.isValidEmail()` que debería ser un Custom Validator.

- En el `LoginController` se realiza un manejo de sesión HTTP manualmente cuando Spring Security debería encargarse de esto.
- Siguen utilizando `@Preauthorize` en la capa de servicios. **Esto ya había sido marcado en la primer entrega.**
- Les siguen faltando anotaciones `@Transactional` en algunos métodos de los servicios, como `deleteCalendarEvent()`. **Esto ya había sido marcado en la primer entrega.**
- Hacen uso de `FetchType.EAGER` en algunos modelos
 - Incluso con valores máximos como 10 IDs de imágenes, cargando ya 10 ofertas en una lista paginada se pueden ejecutar cargar 20 ofertas en una lista paginada, se pueden ejecutar ~200 queries en lugar de 1. **Esto es un error conceptual grave.**
- Realizan operaciones de filtrado y agregación en Java que deberían ser SQL.

Seguimiento

- No resta puntos por seguimiento.

Nota

5; promedio 5.5

Grupo 12

Demo

- Los iconos de categoría no están centrados en pantalla
- La mayoría de los emprendimientos listados dicen "No estar disponible en este momento"
 - Como usuario es rarísimo, pero encima es innecesario. Estos vendedores simplemente no tienen datos de pago, pero eso sólo es necesario al aceptarse la venta. El flujo debiera llevar a que el vendedor cargue los datos antes de aceptar la venta, no a bloquear al comprador.

Código

- En la entrega anterior se les marcó:

El método `ensureAcceptedBelongsToRequester` y `ensurePendingBelongsToSeller` suena a que realiza lógica verificación de "ownership" de recursos. Esta lógica debería ser ejecutada por spring security y no estar directamente en el controller (en el controller si lo necesitan seria simplemente obtener el dato)

Esto no ha sido corregido.

- Concatenan textos en el fornt en lugar de interpolar mensajes. Esto asume que el orden de los elementos no es dependiente del idioma, lo que es falso. **Esto es un error grave.**
- Los tests no son unitarios, haciendo uso de múltiples operaciones distintas durante cada operación. Esto es incluso reflejado en el propio nombre de los tests, con cosas como `create_and_findById_ok`. **Esto es un error conceptual grave.**
- Los tests de persistencia en general no están validando el estado de la base de datos, sólo las respuesta del método. **Esto es un error conceptual grave.**
- El código evita intencionalmente chequeos de tipos. Para enums como `Province` o `Category` no sólo hay métodos que los reciben como `String`, sino que quien los invoca muchas veces convierte de enum a `String` en forma proactiva. **Esto es un error grave.**

Seguimiento

- No resta puntos por seguimiento.

Nota

4; promedio 6

Grupo 13

Demo

- Tienen mensajes de longitud invertidos (primero el max y luego el min)

- Si bien se valida luego, los inputs numéricos aceptan negativos cuando no es un valor valido
- Quizás 200 caracteres para la descripción es poco
- El copy link copia el nombre del evento en lugar del link
- No puedo cambiar mi contraseña o establecer una foto de perfil
- En la vista de evento, estaría bueno mostrar mas info de los organizadores, como por ejemplo followers, follows, cantidad promedio de asistencia
- Las acciones destructivas no tienen un modal de confirmación
- Como asistente nunca me entero si mi postulación al evento fue aceptada o no
- En el mail de notificaciones falta el titulo de la notificación
- Las notificaciones se muestran de manera inconsistente. A veces muestra titulo y descripción y a veces no
- Deberían mostrar fecha de los mensajes
- Si un organizador entra a su evento, directamente mostrarle todas las opciones. Ir al evento desde el home o desde el perfil lleva al organizador a vistas diferentes
- Para ver el historial de notificaciones, tengo que clickear en send a notification
- Estaría bueno poder recibir un recordatorio cuando se acerca un evento

Código

- Se crea un bean para `CharacterEncodingFilter` que luego no se usa, el filtro está configurado directamente en el `web.xml` . **Esto ya se marcó en la primera entrega.**
- Definen controllers con switch cases que utilizan magic strings para definir qué operación realizar, en lugar de definir un endpoint apropiado para cada una.
- En Tags tienen como atributo la cantidad de eventos con ese tag, lo que los obliga a manejar la consistencia de dicho valor, volviéndose propenso a errores y complejizando el código. Una forma simple de manejarlo sería a

través de `@Formula` y no guardarlo en base de datos. Lo mismo sucede con Presenter.

- Tienen tests no unitarios, llamando multiples veces a la clase under test.
Esto es un error conceptual.

Seguimiento

- Las stories no estan etiquetadas por el tipo (feature, chore, bug)
- No resta puntos por seguimiento.

Nota

6; promedio 6

Grupo 14

Demo

- La pagina no funciona en ingles - deployaron algo mal y no cargaba toda la pagina
- Faltaría exponer la cantidad de calificaciones de un usuario
- Estaría bueno poder decidir irte de un equipo
- Limitar la sugerencia de usuarios porque puede ir hasta el infinito
- La pagina tiene mucha funcionalidad y muy compleja. La UI se atiene bien al publico deseado.

Código

- Hacen uso de `FetchType.EAGER` en algunos modelos
 - Por ejemplo, un `Game` trae SIEMPRE todos sus formatos, incluso cuando no se necesitan. Aunque `GameFormat` no es una cardinalidad extremadamente alta, no hay justificación para EAGER. Se pueden llegar a ejecutar muchas queries si se hace un pedido muy grande de Games. **Esto es un error conceptual grave.**
- Se hace un llamado a `.size()` en una colección Lazy. Esto materializará toda la colección. Si se necesita solo la cantidad de elementos, debería usarse

un `COUNT` en la query. Más aún, no parece utilizarse este valor en absoluto.

- Se realizan varias queries sin paginación. **Esto es un error grave.**
 - Se traen todos los participantes de un torneo en `ParticipantHibernateDao` (`getTournamentParticipantUsers`).
 - También se ve en otros métodos de `GameHibernateDao` como `getUserTeams()` y `searchByGenre()`.
- Falta mucha cobertura de tests, particularmente del servicio `TournamentService`. Falta cobertura para métodos complejos como `setFinished`, `startTournament`, etc que contienen extensa lógica de negocio. **Esto ya había sido marcado en la primer entrega.**

Seguimiento

- No resta puntos por seguimiento.

Nota

6; promedio 6.5

Grupo 15

Demo

- El mail de validación no tiene un link para validar directamente, siempre hay que copiar y pegar el código a mano.
- El mail de oferta aceptada no incluye los datos de pago y monto a pagar, sólo son accesibles desde la web.
- "Aceptar y eliminar publicación" está en rojo... pero la operación es positiva (aceptar), el código de colores no es correcto.
- Revisar delta de funcionalidad vs entrega anterior.

Código

- Omiten modificadores de acceso y no parece ser por diseño.
- Crean algunas queries bastante ineficientes, por ejemplo usando múltiples (y repetidas) queries anidadas.

- Tienen varios métodos auxiliares para generar queries complejas, pero siendo estos privados, no tienen forma de testearse salvo indirectamente. Dado lo limitados de los tests sobre los métodos públicos relevantes (presencia / ausencia de filtros opcionales), la cobertura en esta parte crítica del código resulta insuficiente.
 - Estos métodos auxiliares, aunque independientes, en la práctica están totalmente acoplados, debiendo conocer y setear todos los mismos nombres de parámetros; lo que refuerza la necesidad de testing y la fragilidad de la implementación actual.

Seguimiento

- No resta puntos por seguimiento.

Nota

8; promedio 8